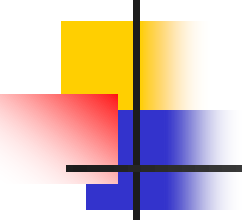


Introducción a la computación

SESION 11

Contadores y Acumuladores

UNIVERSIDAD NACIONAL DE INGENIERIA
Facultad de Ingeniería Industrial y de Sistemas
Ing. Miguel Angel Navarro Neyra
mnavarron@uni.edu.pe

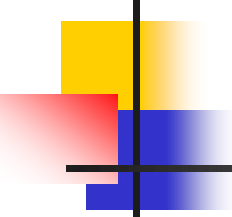


***¿En qué situaciones de la vida
diaria creen que se utilizan
contadores o acumuladores?***

¿Cuidado con el OCD?



Revisar: aritmomanía



Agenda

Plan de Actividades

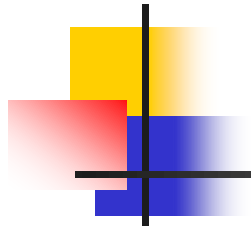
Conceptos

Implementación en C++

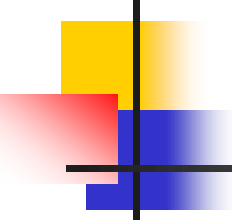
Ejemplo guiados

Errores communes y buenas
prácticas

Ejercicios



Conceptos



¿Que es un contador?

Un contador es una variable que se incrementa (o decrementa) cada vez que ocurre un evento determinado. Su función principal es contar cuántas veces ocurre algo.

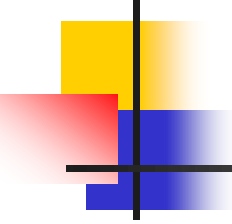
Características:

- Se inicializa normalmente en 0.
- Se incrementa con:
 contador = contador + 1; o
 contador++; o
 ++contador;
- Es de tipo int en la mayoría de los casos.

```
int numero, contador = 0;

for (int i = 0; i < 5; i++) {
    cout << "Ingrese un número: ";
    cin >> numero;
    if (numero > 0) {
        contador++;
    }
}

cout << "Cantidad de números positivos: ";
cout << contador << endl;
```



¿Que es un acumulador?

Un acumulador es una variable que suma (o acumula) valores durante la ejecución de un bucle. Se usa para obtener totales o promedios.

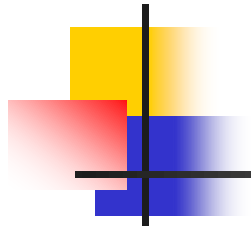
✎ Características:

- Se inicializa en 0 (para suma) o 1 (para multiplicación).
- Se actualiza con:
acumulador = acumulador + valor; o
acumulador += valor;
- Puede ser int, float o double, según lo que se acumule.

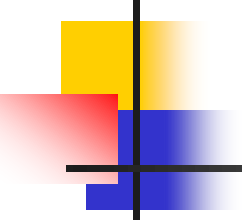
```
int numero, suma = 0;

for (int i = 0; i < 5; i++) {
    cout << "Ingrese un número: ";
    cin >> numero;
    suma += numero;
}

cout << "Suma total: " << suma << endl;
```



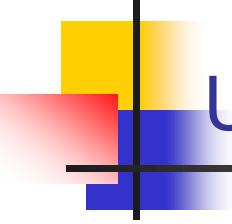
Implementación en C++



Declaración e inicialización de variables

Antes de usarlos, debes declarar e inicializar:

```
int contador = 0;    // contador se usa para contar ocurrencias
int suma = 0;        // acumulador para sumar valores
```



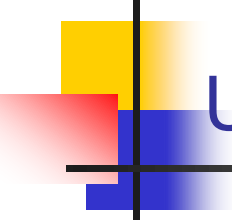
Uso del contador (incremento o decremento)

Un contador incrementa su valor cuando ocurre un evento deseado:

```
for (int i = 0; i < 10; i++) {  
    if (i % 2 == 0) {  
        contador++; // cuenta los números pares del 0 al 9  
    }  
}
```

Alternativas:

```
contador = contador + 1;  
contador += 1;  
++contador; // pre-incremento
```

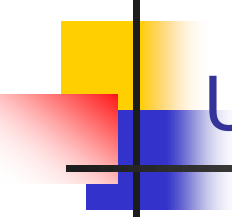


Uso del acumulador (sumar o acumular valores)

El acumulador se usa para sumar valores ingresados:

```
int nota, suma = 0;

for (int i = 0; i < 5; i++) {
    cout << "Ingrese una nota: ";
    cin >> nota;
    suma += nota; // acumulamos las notas
}
```



Uso combinado (contador + acumulador)

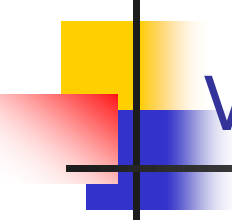
Se usa cuando quieres contar y acumular al mismo tiempo, por ejemplo, para calcular un promedio:

```
int numero, suma = 0, contador = 0;

cout << "Ingrese números positivos (-1 para terminar):" << endl;
while (true) {
    cin >> numero;
    if (numero == -1) break;

    suma += numero;
    contador++;
}

if (contador > 0) {
    float promedio = static_cast<float>(suma) / contador;
    cout << "Promedio: " << promedio << endl;
} else {
    cout << "No se ingresaron valores válidos." << endl;
}
```

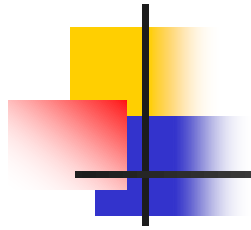


Validación y protección contra errores

Siempre valida el ingreso de datos y evita división por cero:

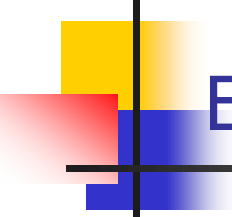
```
if (contador != 0) {  
    float promedio = static_cast<float>(suma) / contador;  
}
```

También puedes usar condiciones dentro del bucle para evitar acumular datos incorrectos.



Ejemplos guiados

Pre-incremento y Post-
incremento en C++



Ejemplo 1: Asignación simple

```
#include <iostream>
using namespace std;

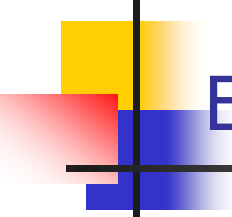
int main() {
    int i = 5;
    int a = ++i;  // pre-incremento
    int b = i++;  // post-incremento

    cout << "Valor de i: " << i << endl; // 7
    cout << "Valor de a (pre-incremento): " << a << endl; // 6
    cout << "Valor de b (post-incremento): " << b << endl; // 6
    return 0;
}
```

🔗 Explicación:

++i: i se incrementa antes, así que a = 6.

i++: b toma primero el valor 6, luego i pasa a 7.



Ejemplo 2: En expresiones

```
#include <iostream>
using namespace std;

int main() {
    int i = 3;
    int resultado1 = ++i + 5;
    int resultado2 = i++ + 5;

    cout << "Resultado con ++i: " << resultado1 << endl; // 9
    cout << "Resultado con i++: " << resultado2 << endl; // 9
    cout << "Valor final de i: " << i << endl; // 5
    return 0;
}
```

✎ el valor usado en la expresión depende de si el incremento fue antes (++i) o después (i++).



Ejemplo 3: En un bucle for

📌 Conclusión: En bucles simples como for, no hay diferencia visible en el resultado entre ++i e i++. Pero se recomienda ++i por buena práctica de eficiencia, especialmente con iteradores.

```
#include <iostream>
using namespace std;

int main() {
    cout << "Usando ++i:\n";
    for (int i = 0; i < 5; ++i) {
        cout << i << " ";
    }

    cout << "\nUsando i++:\n";
    for (int i = 0; i < 5; i++) {
        cout << i << " ";
    }

    return 0;
}
```



Ejemplo 4: Aplicación con acumulador

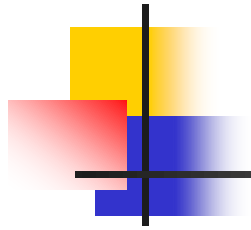
```
#include <iostream>
using namespace std;

int main() {
    int i = 1;
    int suma1 = 0, suma2 = 0;

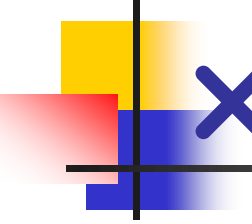
    while (i <= 3) {
        suma1 += ++i; // pre-incremento
    }

    i = 1; // reiniciar i
    while (i <= 3) {
        suma2 += i++; // post-incremento
    }

    cout << "Suma con ++i: " << suma1 << endl; // 9 (2+3+4)
    cout << "Suma con i++: " << suma2 << endl; // 6 (1+2+3)
    return 0;
}
```



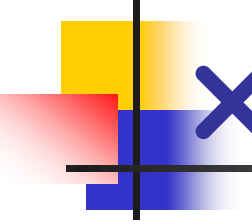
Errores comunes y buenas prácticas



✗ Usar i++ sin entender el orden de evaluación

```
int i = 5;  
int x = i++ + 1; // x será 6, no 7
```

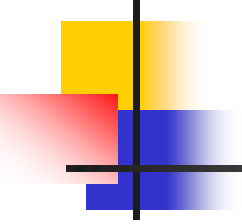
📌 Error: Pensar que i++ ya incrementó el valor usado en la suma.



✗ No inicializar los contadores o acumuladores

```
int suma; // sin inicializar
for (int i = 0; i < 5; i++) {
    suma += i; // ✗ Comportamiento indefinido
}
```

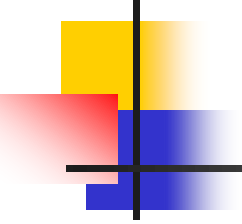
📌 Solución: Siempre inicializar: `int suma = 0;`



✗ Modificar el mismo contador dos veces en la misma expresión

```
int i = 1;  
// ✗ comportamiento indefinido (orden de evaluación ambigua)  
int x = i++ + ++i;
```

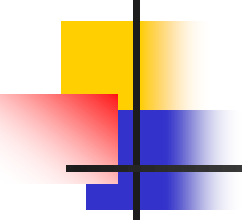
📌 Regla: Nunca combines post y pre-incrementos sobre la misma variable en una misma expresión.



✗ Olvidar actualizar el contador/acumulador en ciclos while

```
int i = 0;
while (i < 5) {
    cout << i << endl;
    // ✗ Falta i++ → ciclo infinito
}
```

🔗 Solución: Asegurar siempre una condición de salida y la actualización correspondiente.



✗ Dividir por cero al calcular promedios

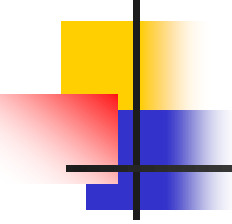
```
float promedio = suma / contador; // ✗ Si contador = 0 → error
```

📌 Solución:

```
if (contador != 0)
    promedio = static_cast<float>(suma) / contador;
```




Ejercicios



Ejercicio 01

✎ Dado un número entero positivo ingresado por el usuario, contar cuántos dígitos son pares y cuántos impares. Mostrar ambos resultados.

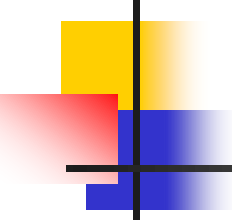
Ejemplo: Entrada: 4829 → Salida: pares = 2, impares = 2

Puntos clave:

Uso de while y operadores % y /

Contadores múltiples

Validación de entrada



Ejercicio 02

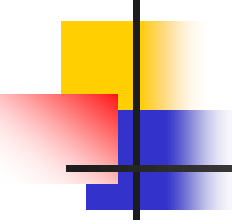
✎ Solicita al usuario ingresar n números reales. Solo se deben acumular los valores positivos. Calcular y mostrar el promedio de esos positivos. Validar que n sea mayor que 0.

Ejemplo: Entrada: 5, luego: -3, 10, 4.5, -2, 6 $\rightarrow$ promedio = $(10+4.5+6)/3 = 6.83$

Puntos clave:

Acumulador y contador bajo condición

Validación de entrada y división segura



Ejercicio 03

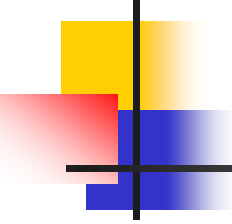
✎ Pedir al usuario un número entero N (>1) y mostrar todos los números primos entre 1 y N , además de contar cuántos hay.

Puntos clave:

Doble ciclo (for anidado)

Uso de break y condiciones compuestas

Contador de primos



Ejercicio 04

✎ Ingresar 15 edades. Determinar y mostrar:

Cuántos son menores de edad

Cuántos son adultos mayores (≥ 60)

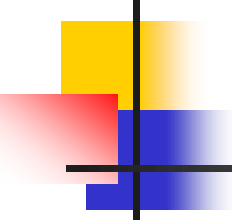
El promedio general de edades

Puntos clave:

for con acumulador y múltiples contadores

Cálculo de promedio

Uso opcional de vector si se desea almacenar



Ejercicio 05

✎ Pedir al usuario ingresar calificaciones hasta que escriba -1.
Ignorar (no contar) las notas menores a 0 o mayores a 20. Al final, mostrar:

El número de notas válidas

El promedio de las válidas

Cuántas notas fueron desaprobadas (<11)

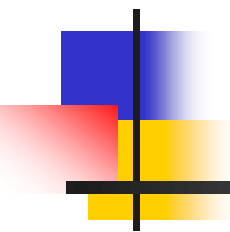
Puntos clave:

Ciclo while (true) con break

Validación con continue

Contadores condicionales y promedio

Introducción a la computación



FIN SESION 11